ICT Academy of Kerala · Python Programming

Student Result Management System

1-hour case study · Trainer solution

Working code, real output and facilitation guide

The scenario

A training centre keeps the marks of its students in a Python **dictionary**: the key is the student id and the value is another dictionary with the student's name and a dictionary of subject marks. The office staff currently work out totals, averages and grades by hand. **You will turn that work into functions** so the centre can calculate results, update marks, add and remove students, and print a report with one call.

File name: `result_system.py` **Time:** 60 minutes

What you will practise

- Dictionaries: nested dictionaries, `.values()`, `.items()`, `.update()`, `.pop()`, `.setdefault()`
- Loops, `if / elif / else` and calculating totals and averages
- Functions: parameters, `return`, positional and keyword arguments, default argument (`pass_mark=40`), `*args` (`*subjects`), `**kwargs`, unpacking a dictionary with `**`, functions calling other functions
- Sorting a list of tuples and formatted output with f-strings

STARTER DATA (INPUT IN DICTIONARY FORMAT; COPY EXACTLY)

```
students = {
    "S101": {"name": "Anu",   "marks": {"Python": 88, "Maths": 76, "Stats": 92}},
    "S102": {"name": "Binu",  "marks": {"Python": 45, "Maths": 38, "Stats": 52}},
    "S103": {"name": "Cathy", "marks": {"Python": 95, "Maths": 91, "Stats": 89}},
    "S104": {"name": "Dev",   "marks": {"Python": 60, "Maths": 72, "Stats": 55}},
    "S105": {"name": "Esha",  "marks": {"Python": 72, "Maths": 65, "Stats": 80}},
}
```

Functions to write

Function	What it does
calculate_total(marks)	returns the sum of all marks in the marks dictionary
calculate_average(marks)	returns the average rounded to 2 decimals (use `calculate_total`)
get_grade(average)	returns `"A"` for 90 and above, `"B"` for 75 and above, `"C"` for 60 and above, `"D"` for 40 and above, otherwise `"F"`
check_result(marks, pass_mark=40)	returns `"Pass"` only if **every** subject is at least the pass mark, otherwise `"Fail"`
student_summary(student)	takes one student dictionary and returns a new dictionary with name, total, average, grade and result
add_student(data, student_id, name, **marks)	adds a student with the marks given as keyword arguments and returns `True`; if the id already exists it changes nothing and returns `False`
update_marks(data, student_id, **changes)	updates the marks given as keyword arguments (for example `Maths=55`) and returns `True`; returns `False` if the id does not exist
remove_student(data, student_id)	removes the student and returns `True`; returns `False` if the id does not exist
print_report(data)	prints the report table, students sorted by average, highest first (build a list of `(average, id)` tuples and use `sort(reverse=True)`)
class_statistics(data, *subjects)	returns a dictionary with the topper's name, the pass percentage (1 decimal) and a dictionary of the average mark of each subject (1 decimal). Extra **positional** arguments (`*subjects`) limit the subject averages to those subjects; with none given, use all subjects
main()	prints each task heading and calls the functions; end the file with `main()`

Every function should **return** its result (except `print_report` and `main`, which print). Do not use global variables inside the functions; pass the dictionary in as a parameter.

Tasks and expected output

Task 1 · Summary of one student

Print the summary dictionary of student S101. Then print `check_result` for the marks of S102 with the default pass mark, with `pass_mark=35` as a **keyword** argument, and once more passing `35` as a **positional** argument (the three results show the difference is only how you write the call).

EXPECTED OUTPUT

```
Task 1: One student
{'name': 'Anu', 'total': 256, 'average': 85.33, 'grade': 'B', 'result': 'Pass'}
Fail
Pass
Pass
```

Task 2 · Report of the class

Print the report of all students. Widths: ID 6, Name 8, Total 6, Avg 8 (two decimals), Grade 7, then two spaces and the result. A line of 42 dashes under the heading. Sort by average, highest first: collect `(average, id)` tuples in a list and call `sort(reverse=True)`.

EXPECTED OUTPUT

```
Task 2: Report
ID    Name     Total     Avg  Grade  Result
------------------------------------------
S103  Cathy      275   91.67      A  Pass
S101  Anu        256   85.33      B  Pass
S105  Esha       217   72.33      C  Pass
S104  Dev        187   62.33      C  Pass
S102  Binu       135   45.00      D  Fail
```

Task 3 · Update marks

Binu has re-checked his Maths paper and the mark is corrected to 55. Update it with `update_marks` and print the result. Try to update the unknown id S999 and print the result. Then a second set of corrections arrives as a dictionary, `corrections = {"Python": 50, "Stats": 58}`: apply it with **dictionary unpacking** (`**corrections`) and print the result. Print the new summary of S102.

EXPECTED OUTPUT

```
Task 3: Update marks
True
False
True
{'name': 'Binu', 'total': 163, 'average': 54.33, 'grade': 'D', 'result': 'Pass'}
```

Task 4 · Add a student

Add a new student S106, Farhan, with Python 70, Maths 35 and Stats 74 (use keyword arguments) and print the result. Try to add S101 again with the name Zara and print the result. Print the new student's dictionary.

EXPECTED OUTPUT

```
Task 4: Add student
True
False
{'name': 'Farhan', 'marks': {'Python': 70, 'Maths': 35, 'Stats': 74}}
```

Task 5 · Remove a student

Remove S104 (Dev) and print the result. Try to remove the same id again and print the result. Print how many students are left.

EXPECTED OUTPUT

```
Task 5: Remove student
True
False
5
```

Task 6 · Final report and class statistics

Print the report again, then use `class_statistics` to print the topper, the pass percentage and the subject averages. Finally call `class_statistics(students, "Python", "Stats")` to print the averages of only those two subjects.

EXPECTED OUTPUT

```
Task 6: Final report and statistics
ID    Name     Total     Avg  Grade  Result
-----------------------------------------
S103  Cathy      275   91.67      A  Pass
S101  Anu        256   85.33      B  Pass
S105  Esha       217   72.33      C  Pass
S106  Farhan     179   59.67      D  Fail
S102  Binu       163   54.33      D  Pass
Topper: Cathy
Pass percentage: 80.0
Subject averages: {'Python': 75.0, 'Maths': 64.4, 'Stats': 78.6}
Python and Stats only: {'Python': 75.0, 'Stats': 78.6}
```

Print the heading lines (for example `Task 1: One student`) exactly as shown, with a blank line between tasks. The tasks run in order, so each task changes the data used by the next one. Write and run one function at a time.

Complete solution

```
# result_system.py - Student Result Management System
students = {
    "S101": {"name": "Anu",   "marks": {"Python": 88, "Maths": 76, "Stats": 92}},
    "S102": {"name": "Binu",  "marks": {"Python": 45, "Maths": 38, "Stats": 52}},
    "S103": {"name": "Cathy", "marks": {"Python": 95, "Maths": 91, "Stats": 89}},
    "S104": {"name": "Dev",   "marks": {"Python": 60, "Maths": 72, "Stats": 55}},
    "S105": {"name": "Esha",  "marks": {"Python": 72, "Maths": 65, "Stats": 80}},
}


def calculate_total(marks):
    return sum(marks.values())


def calculate_average(marks):
    return round(calculate_total(marks) / len(marks), 2)


def get_grade(average):
    if average >= 90:
        return "A"
    elif average >= 75:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 40:
        return "D"
    return "F"


def check_result(marks, pass_mark=40):
    for mark in marks.values():
        if mark < pass_mark:
            return "Fail"
    return "Pass"


def student_summary(student):
    marks = student["marks"]
    average = calculate_average(marks)
    return {"name": student["name"], "total": calculate_total(marks),
            "average": average, "grade": get_grade(average),
            "result": check_result(marks)}


def add_student(data, student_id, name, **marks):
    if student_id in data:
        return False
    data[student_id] = {"name": name, "marks": dict(marks)}
    return True


def update_marks(data, student_id, **changes):
    if student_id not in data:
        return False
    data[student_id]["marks"].update(changes)
    return True


def remove_student(data, student_id):
```

```
    return data.pop(student_id, None) is not None


def print_report(data):
    print(f"{'ID':<6}{'Name':<8}{'Total':>6}{'Avg':>8}{'Grade':>7}  Result")
    print("-" * 42)
    ranking = []
    for sid, student in data.items():
        ranking.append((student_summary(student)["average"], sid))
    ranking.sort(reverse=True)
    for average, sid in ranking:
        s = student_summary(data[sid])
        print(f"{sid:<6}{s['name']:<8}{s['total']:>6}{s['average']:>8.2f}{s['grade']:>7}
{s['result']}")


def class_statistics(data, *subjects):
    summaries = [student_summary(s) for s in data.values()]
    passed = sum(1 for s in summaries if s["result"] == "Pass")
    by_subject = {}
    for student in data.values():
        for subject, mark in student["marks"].items():
            if len(subjects) == 0 or subject in subjects:
                by_subject.setdefault(subject, []).append(mark)
    subject_avg = {sub: round(sum(m) / len(m), 1) for sub, m in by_subject.items()}
    topper = summaries[0]
    for s in summaries:
        if s["average"] > topper["average"]:
            topper = s
    return {"topper": topper["name"], "pass_percent": round(passed / len(summaries) * 100,
1),
            "subject_average": subject_avg}


def main():
    print("Task 1: One student")
    print(student_summary(students["S101"]))
    print(check_result(students["S102"]["marks"]))
    print(check_result(students["S102"]["marks"], pass_mark=35))
    print(check_result(students["S102"]["marks"], 35))
    print("\nTask 2: Report")
    print_report(students)
    print("\nTask 3: Update marks")
    print(update_marks(students, "S102", Maths=55))
    print(update_marks(students, "S999", Maths=55))
    corrections = {"Python": 50, "Stats": 58}
    print(update_marks(students, "S102", **corrections))
    print(student_summary(students["S102"]))
    print("\nTask 4: Add student")
    print(add_student(students, "S106", "Farhan", Python=70, Maths=35, Stats=74))
    print(add_student(students, "S101", "Zara", Python=1))
    print(students["S106"])
    print("\nTask 5: Remove student")
    print(remove_student(students, "S104"))
    print(remove_student(students, "S104"))
    print(len(students))
    print("\nTask 6: Final report and statistics")
    print_report(students)
    stats = class_statistics(students)
    print("Topper:", stats["topper"])
    print("Pass percentage:", stats["pass_percent"])
    print("Subject averages:", stats["subject_average"])
    print("Python and Stats only:", class_statistics(students, "Python", "Stats")
["subject_average"])
```

```
main()
```

Facilitation guide

Function / task	Concepts checked	Common mistakes to watch for
calculate_total / calculate_average	`values()`, `sum`, `len`, return	printing instead of returning; dividing by a fixed 3
get_grade	if / elif / else with returns	using separate `if`s so a grade is overwritten; wrong order of the conditions
check_result	loop with early return, default argument	returning "Pass" inside the loop; comparing the average instead of each subject
student_summary	functions calling functions, building a dictionary	recalculating by hand instead of calling the earlier functions
add_student	`**kwargs`, `in` check, returning True/False	forgetting `dict(marks)`; overwriting an existing id
update_marks	`update()` on the nested dictionary	replacing the whole marks dictionary; updating a missing id (KeyError)
remove_student	`pop(key, None)`	KeyError when the id is missing
print_report	tuples in a list, `sort(reverse=True)`, f-string widths	sorting the ids instead of the averages; misaligned columns
class_statistics	`setdefault`, dictionary comprehension, finding the largest with a loop	wrong denominator for the pass percentage

Quick marking checklist

- Output matches exactly (spacing and alignment)
- Functions return values; no global variables inside functions
- Default argument, `*args` and `**kwargs` used as specified
- Nested dictionary updated in place (not rebuilt)
- Readable code: meaningful names, comments, no copy-paste blocks

Discussion questions for the debrief

1. Why does Binu's result change from Fail to Pass after the update, although his grade stays D?
2. Why do `add_student` and `update_marks` change the dictionary that was passed in, but `student_summary` does not?
3. Which function would you change if the pass mark became 45 for the whole centre? Which if only one exam needed it?

Trainer tip: Demo `calculate_total`, `calculate_average` and `get_grade` live (8 minutes). Pause after Task 2 and let pairs compare their reports before they start changing the data.